



G

O

D

O

O

O

O

D

Quiz 1: NachOS

scheduler.cc
 Threads.cc
 ThreadTest.cc → test procedure called from main()

Scheduler: maintains a list of threads that are ready to run
 → maintains ready list/queue
 → Invoked whenever current thread gives up the CPU (NON-preemptive)

Simple scheduling policy:

1. Equal priority
2. FCFS
3. Append end, remove front

Thread:

States:

- READY: eligible to run
- RUNNING: only one thread can be in this state
- BLOCKED: waiting for some external event
- JUST_CREATED: temporary state used during creation

Fork(function, arg, flag) → ready

- Turns a thread into one that can be executed
- Calls readyToRun() → put thread into ready list

Yield() → to push into ready queue & run thread

- Finds the next thread to run using findNextToRun()
- If another thread has been found
 - Call readyToRun() for the old thread
 - Run the new thread using Run()

Sleep()

- Set status to **BLOCKED** → waiting
- Find another thread to run using findNextToRun()
- If another thread has been found
 - Run the new thread using Run()

Finish() → exit

- Called at the end of execution
- Marks a thread for termination

SimpleThread() function:

```

void SimpleThread(int which) {
    int num;
    for (num = 0; num < 3; num++) {
        printf("**** thread %d looped %d times\n", (int) which, num);
        currentThread->Yield(&num);
    }
    // 1. the first thread in the ready q is fetched. (nextThread = scheduler->indexToRun(j))
    // 2. current thread is put back to end of ready queue (scheduler->readyToRun(this); , see scheduler.cc)
    // 3. the fetched thread is run. (scheduler->Run(nextThread));
    // repeat until each thread's simplethread function has run 3 times.
  }
  
```

Remember the only argument passed by fork? This is it. Here: 'which' tells you which thread

→ does nothing except print status line 3 times

ReadyToRun(thread)

- Makes a thread ready to run (set state to READY)
- Adds it to the ready list
- Does not start the thread yet!

FindNextToRun(): thread

- Simply returns the thread at the front of the ready list

Run(thread)

- Switches from one thread to another:
 1. Check whether the current thread overflowed its stack
 2. Change the state of the new thread to **RUNNING**
 3. Perform the actual context switch (by calling Switch())
 4. Terminate previous thread (if applicable)

Directory Structure

bin	For generating NachOS format files, DO NOT CHANGE!
filesystems	NachOS kernel related to file system, DO NOT CHANGE!
exp1	Experiment 1, nachos threads.
exp2	Experiment 2, CPU scheduling.
machine	MIPS HWW simulation, DO NOT CHANGE unless asked.
Makefile.common	For compilation of NachOS, DO NOT CHANGE!
Makefile.dep	
network	NachOS kernel related to network, DO NOT CHANGE!
port	NachOS kernel related to port, DO NOT CHANGE!
readme	Short description of OS labs and assessments
test	NachOS format files for testing virtual memory, DO NOT CHANGE!
threads	NachOS kernel related to thread management, DO NOT CHANGE!
userprog	NachOS kernel related to running user applications, DO NOT CHANGE!

ThreadTest function:

```

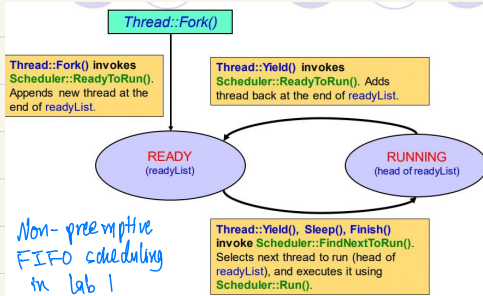
void ThreadTest() {
    DEBUG('t', "Entering SimpleThread");
    Thread *t1 = new Thread("child1");
    t1->Fork(SimpleThread, 1, 0);
    Thread *t2 = new Thread("child2");
    t2->Fork(SimpleThread, 2, 0);
    SimpleThread(0);
}
  
```

→ sets thread to **JUST_CREATED** status
 Thread constructor does only minimal initialization
 Turns a thread into one that can be scheduled and executed by the CPU
 After starting two new threads, execute the SimpleThread function

Fork(function, arg, join) → thread id; only argument passed in

- "function": the procedure to be executed concurrently
 - Here: SimpleThread
- "arg": a single parameter which is passed to the procedure
 - Here: the id of the thread (0,1,2)
- "join": a flag which indicates whether the thread can be joined
 - Not important for the lab

Big Picture of Thread & Scheduler:



Threads detailed functionalities:

Thread()

- Constructor: sets the thread as **JUST_CREATED** status

Fork()

- Allocate stack, initialize registers.
- Call **Scheduler::ReadyToRun()** to put the thread into **readyList**, and set its status as **READY**.

Yield()

- Suspend the calling thread and put it into **readyList**.
- Call **Scheduler::FindNextToRun()** to select another thread from **readyList**.
- Execute selected thread by **Scheduler::Run()**, which sets its status as **RUNNING** and call **SWITCH()** (in **code/threads/switch.s**) to exchange the running thread.

Finish()

- Mark current thread for destruction.
- Call **Sleep()** to find next thread to run and execute it.

void Sleep()

- Suspend the current thread and change its state to **BLOCKED**
 - Run next ready thread
 - Invoke **Interrupt->Idle()** to wait for the next interrupt when **readyList** is empty
- Sleep** is called when the current thread needs to be blocked until some future event takes place.
 - Eg. Waiting for a disk read interrupt
 - It is called by **Semaphore::P()** in **code/threads/synch.cc**.
 - Semaphore::V()** will wake up one of the thread in the waiting queue (sleeping threads queue).

bool CheckIfDue()

- Function to process interrupts and invoke handler.
- Checks if **pendingInterrupt** at the head of **pending list** should be triggered at current tick.
- If yes, corresponding handler is invoked.
- Handler for **Timer** is **Timer::TimerExpired()**.

void YieldOnReturn()

- Function that is called by the **Timer handler** **TimerInterruptHandler()**.
- Sets the variable **yieldOnReturn** to true.
- Force **Interrupt::OneTick()** to trigger context switch.

Timers:

→ was several functions from interrupt class
→ Used to trigger an interrupt (fixed no. of ticks)

void TimerInterruptHandler()

- Function defined in **code/threads/system.cc**.
- Executes whenever the associated timer expires and the interrupt is triggered.
- Timer** is initialized in **code/threads/system.cc** using the constructor for class **Timer** which is defined in **code/machine/timer.cc**.

void TimerExpired()

- Function defined in **code/machine/timer.cc**.
- Executes whenever the timer expires. It in turn invokes the interrupt handler which is defined in previous slide.

int TimeOfNextInterrupt()

- Defined in **code/machine/timer.cc**.
- Returns an integer denoting number of time ticks.
- Used to schedule an interrupt using the timer. The interrupt will be triggered after this number of time ticks from the current time.
- Can be used to make the timer periodic as required for round-robin scheduling.

Interrupt:

- Pending timer interrupts in system are maintained in list called "pending" from class "PendingInterrupt".
- List sorted in increasing order of time ticks when interrupt will be triggered

void Schedule()

- Function schedules/inserts a new interrupt to the **pending list**.
- Insertion is in **sorted** order; sorted by the pending time ticks for the interrupt to be triggered.
- Used in **Timer** to initialize a timer interrupt.

void OneTick()

- Function to process a single time tick.
- Updates global variable **stats** → **totalTicks**.
- Calls **Interrupt::CheckIfDue()** (defined below) to process any pending interrupt that would be triggered now.
- If variable **yieldOnReturn** is true, then triggers a context switch through a call to **Thread::Yield()**.

Lab 2: (a)

```
// CODE TO BE EDITED

//Under system.cc, set randomYield to TRUE:
bool randomYield = TRUE; // timer is activated by this? i changed this.
// This will permanently activate the function we need in Timer.cc

// Next, in Timer.cc copy this section in Timer() into TimerExpired().
interrupt->Schedule(TimerHandler, (_int) this, TimeOfNextInterrupt(),
    TimerInt);
// This will ensure TimerExpired will always schedule the next interrupt after a current interrupt is invoked..

// Finally, under TimeOfNextInterrupt(), set return to permanently 40, or comment out the section under if(randomize) and set it to permanently return 40, like so:
if (randomize)
    return 40; // 1 + (Random() % (TimerTicks * 2)); <- original, changed to return 40
// since randomize = true for timer to be activated
else
    return TimerTicks;
// This ensures, since random is set to True permanently, that 40 ticks will always be returned when timer expires in an interrupt invocation.
```

b)

```
// Under threads.cc, find the finish() function portion, then:
// Step 1. add in this code chunk after #ifdef CHANGED:
#ifdef CHANGED
static void TimerHandler(_int arg)
{ Timer *p = (Timer *)arg; p->TimerExpired(); }
// This is a copy of a function that we will need to use.

// Step 2. Under the finish() function, under both if else results, insert this one line above sleep():
(void) interrupt->Schedule(TimerHandler, (_int)timer ,400,TimerInt);
// This snippet will tell the universal interrupt pointer (interrupt->) to :
// 1. Feed the universal timer pointer (timer) to TimerHandler().
// 2. TimerHandler() (using universal timer pointer) signals that the current interrupt is expired and due to be replaced.
// 3. That the next interrupt is to be 400 seconds from now, and that it is Type TimerInt (you may simply set this to 40 seconds instead)

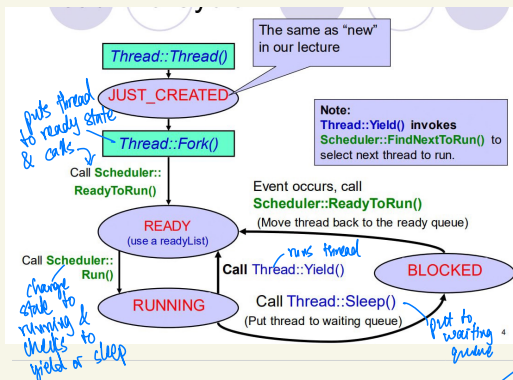
// Under interrupt.cc, check the schedule() function:
// Add this snippet if you used 400 seconds from now:
if(fromNow == 400){
    delete pending->Remove();
    fromNow = fromNow / 10;} // '400' signifies that this schedule call was from a thread finishing, and thus must be corrected.

// Add this if you used 40 seconds:
delete pending->Remove(); // This deletes the present scheduled interrupt.

// The rest of the schedule() function will schedule a new interrupt at present time + 40, thus fulfilling our requirements.
```

Quiz 2:

Thread Life Cycle:



- Thread()**
 - Constructor: sets the thread as **JUST_CREATED** status
- Fork()**
 - Allocate stack, initialize registers.
 - Call **Scheduler::ReadyToRun()** to put the thread into **readyList**, and set its status as **READY**.
- Yield()**
 - Suspend the calling thread and put it into **readyList**.
 - Call **Scheduler::FindNextToRun()** to select another thread from **readyList**.
 - Execute selected thread by **Scheduler::Run()**, which sets its status as **RUNNING** and call **SWITCH()** (in `code/threads/switch.s`) to exchange the running thread.
- Sleep()**
 - Suspend the current thread and find other thread to run
 - Change its state to **BLOCKED**.

void Join (Thread *forked)

- The current thread **waits** for specified thread to finish before continuing.
- Argument **forked**
 - Specify the thread to wait for;
 - The thread forked must be a joinable thread (Fork with **joinP=1**).

Thread *Thread(char *debugName)

The Thread constructor

- Setting status to **JUST_CREATED**,
- Initializing stack to **NULL**, and
- Given the thread name for *debugging*.

Fork(VoidFunctionPtr func, int arg, int joinP);

- Thread creation**
 - Allocating stack by invoking **StackAllocate()** function
 - Put this thread into ready queue by calling **Scheduler::ReadyToRun()**
- Argument func**
 - The address of a **procedure** where execution is to begin when the thread starts executing (**The thread's handler function**)
- Argument arg**
 - An integer argument that would be passed to thread handler function
- Argument joinP**
 - Indicate if a Join is going to happen. If **joinP=1**, a join is going to happen.

Semaphores:

- Non-negative integer
- Initial value = no. of available resources

2 Operations:

1. P() (down/wait)

↳ Waits until S value > 0 b4 S--

↳ Thread can enter CS

↳ If value = 0, calling thread put to sleep

2. V() (up/signal)

↳ ↑ S value

↳ Puts 1st thread

in waiting queue to ready list ⇒ scheduler: ReadyToRun()

↳ Called when thread exits CS

Synchronization in MacOS:

3 primitives: 1. Semaphores

2. Lock

3. Condition Variables

↳ threads / synch.h

↳ threads / synch.cc

Locks:

→ Using Binary semaphores

↳ FREE / BUSY value + lock_owner attribute

→ 2 operations:

1. Acquire()

↳ Only 1 thread can acquire lock

↳ If lock busy, other threads to wait

2. Release()

↳ Thread release() lock to be free again

↳ Lock can be acquired by next thread

→ Locks are owned by threads s.t.:

1. Once acquired, a lock has exactly 1 owner

2. Only owner can release lock

Experiment 3:

- Implement race condition scenarios for inconsistent output (task 1)

prevent scenarios where a CS during critical section will not allow another thread to enter its CS

- Implement process synchronization scheme for consistent output (task 2)

bin	For generating NachOS format files, DO NOT CHANGE!
filesystem	NachOS kernel related to file system, DO NOT CHANGE!
exp3	Experiment 3, process synchronization.
machine	MIPS HW simulation, DO NOT CHANGE unless asked.
Makefile.common	For compilation of OS labs and assessments, DO NOT CHANGE!
Makefile.dep	
network	NachOS kernel related to network, DO NOT CHANGE!
port	NachOS kernel related to port, DO NOT CHANGE!
readme	Short description of OS labs and assessments
test	NachOS format files for testing virtual memory, DO NOT CHANGE!
threads	NachOS kernel related to thread management, DO NOT CHANGE!
userprog	NachOS kernel related to running user applications, DO NOT CHANGE!
vm	Experiment 4, coding virtual memory (TLB, page replacement)

- Calling ThreadTest() to test the above scenarios.

Condition Variables:

→ Requires lock to use

→ 3 Operations:

1. Wait (lock)

- Releases lock

- Claims CPU to thread using it

- Re-acquires lock

2. Signal (lock)

- Wakes up 1st thread in the waiting queue

3. Broadcast (lock)

- Wakes up all threads in wait queue

Task 1:

- Testing of context switches causing interleaving execution orders of 2 threads
↳ leads to race condition for interleaving execution orders

- Use of shared variable, value (init. to 0)

- NachOS is uniprocessor-based & interleaving determined by context switches.

↳ Proper synchronization → Code will run

Voluntary CS:

- Thread running calls

yield() or sleep(), causing context switch

↳ implementations of synch. facility (semaphore)

Involuntary CS:

- Inner NachOS

modules decide to switch to another thread → timer interrupt

signals current thread consumed allocated time quantum

Task 1: Shows race condition

TestValueOne
TestValueMinusOne } produces 2 different values of

TestValueOne():

```
//targetV=1;
//After executing TestValueOne(), the value should be one.
//1. implement the new version of Inc: Inc_v1
void Inc_v1(int which)
{
    int a = value;
    a++;
    currentThread->yield();
    value = a;
    printf("**** Inc thread %d new value %d\n", (int) which, value);
}

//2. implement the new version of Dec: Dec_v1
void Dec_v1(int which)
{
    int a = value;
    a--;
    value = a;
    printf("**** Dec thread %d new value %d\n", (int) which, value);
}
```

CS happens here to next thread in queue
join() to resume here.
NO CS

```
//3. Implement TestValueOne by create two threads with Inc_v1 and two threads with Dec_v1
// you should pass the checking at the end, printing "congratulations! passed."
void TestValueOne()
{
    value=0;
    printf("enter TestValueOne, value=%d...\n", value);

    // fill your code here

    // a. define four empty threads
    Thread *t1 = new Thread("Inc_v1_1");
    Thread *t2 = new Thread("Inc_v1_2");
    Thread *t3 = new Thread("Dec_v1_1");
    Thread *t4 = new Thread("Dec_v1_2");

    // b. fork all the four empty threads
    t1->fork(Inc_v1, 1, 1);
    t2->fork(Inc_v1, 2, 1);
    t3->fork(Dec_v1, 1, 1);
    t4->fork(Dec_v1, 4, 1);

    // c. join all the child threads to main(current) thread
    currentThread->join(t1);
    currentThread->join(t2);
    currentThread->join(t3);
    currentThread->join(t4);

    //2. checking the value. you should not modify the code or add any code lines behind
    //this section.
    if(value==1)
        printf("congratulations! passed.\n");
    else
        printf("value=%d, failed.\n", value);
}
```

queue: t1, t2, t3, t4
value: 2, 1, 0, -1, -2
override: 1, -1, -2
race condition

TestValueMinusOne():

```
//targetV=-2;
//After executing TestValueMinusOne(), the value should be -1.
//1. implement the new version of Inc: Inc_v2
void Inc_v2(int which)
{
    int a = value;
    a++;
    currentThread->yield();
    value = a;
    printf("**** Inc thread %d new value %d\n", (int) which, value);
}

//2. implement the new version of Dec: Dec_v2
void Dec_v2(int which)
{
    int a = value;
    a--;
    currentThread->yield();
    value = a;
    printf("**** Dec thread %d new value %d\n", (int) which, value);
}
```

CS happens here to next thread in queue
join()
CS happen here to next thread in queue.

```
//3. Implement TestValueMinusOne by create two threads with Inc_v2 and two threads with Dec_v2
// you should pass the checking at the end, printing "congratulations! passed."
void TestValueMinusOne()
{
    value=0;
    printf("enter TestValueMinusOne, value=%d...\n", value);

    // fill your code

    Thread *t1 = new Thread("Inc_v2_1");
    Thread *t2 = new Thread("Inc_v2_2");
    Thread *t3 = new Thread("Dec_v2_1");
    Thread *t4 = new Thread("Dec_v2_2");

    t1->fork(Inc_v2, 1, 1);
    t2->fork(Inc_v2, 2, 1);
    t3->fork(Dec_v2, 3, 1);
    t4->fork(Dec_v2, 4, 1);

    currentThread->join(t1);
    currentThread->join(t2);
    currentThread->join(t3);
    currentThread->join(t4);

    //2. checking the value. you should not modify the code or add any code lines behind
    //this section.
    if(value==1)
        printf("congratulations! passed.\n");
    else
        printf("value=%d, failed.\n", value);
}
```

queue: t1, t2, t3, t4
value: 2, 1, 0, -1, -2
override: 1, -1, -2
race cond. current thread puts on wait until t1 finishes by continuing.

```
int value=0;
//case 0
void Inc(_int which)
{
    int a=value;
    a++;
    value=a;
    printf("**** Inc thread %d new value %d\n", (int) which, value);
}

void Dec(_int which)
{
    int a=value;
    a--;
    value=a;
    printf("**** Dec thread %d new value %d\n", (int) which, value);
}
```

Task 2:

Use of Semaphore to ensure consistency

↳ shared variable can have consistent value for different interleaving execution orders & diff. folk orders

```
//4. Implement TestValueUnlabeled by create two threads with Inc_Constant and two threads with Dec_Constant."
// you should pass the checking at the end, printing "congratulations! passed."
void TestConsistency()
{
    value=0;
    printf("Enter TestConsistency, value-%d...\n", value);

    //fill your code
    Thread *inc1 = new Thread("inc1_cons");
    inc1->fork(Inc_Constant, 9, 1);

    Thread *inc2 = new Thread("inc2_cons");
    inc2->fork(Inc_Constant, 9, 1);

    Thread *dec1 = new Thread("dec1_cons");
    dec1->fork(Dec_Constant, 11, 1);

    Thread *dec2 = new Thread("dec2_cons");
    dec2->fork(Dec_Constant, 12, 1);

    currentThread->join(inc1);
    currentThread->join(inc2);
    currentThread->join(dec1);
    currentThread->join(dec2);

    //2. checking the value. you should not modify the code or add any code lines behind
    //this section.
    if(value==0)
        printf("congratulations! passed.\n");
    else
        printf("value-%d, failed.\n", value);
}
```

```
Lock* lock = new Lock("LOCK");
```

```
void Inc_Consistent(_int which)
```

{

```
lock->Acquire();
```

```
int a=value;
```

```
a++;
```

```
currentThread->Yield();
```

```
value = a;
```

```
printf("**** Inc thread %d new value %d\n", (int) which, value);
```

```
lock->Release();
```

```
void Dec_Consistent(_int which)
```

{

```
lock->Acquire();
```

```
int a=value;
```

```
currentThread->Yield();
```

```
value = a;
```

```
printf("**** Dec thread %d new value %d\n", (int) which, value);
```

```
lock->Release();
```

Address Translation:

1. TLB: 1 per machine (FIFO)
2. ZPT: 1 for entire system (LRU)

```
Call Translate(vpn, &phyAddr ...)  
(defined under machine/translate.cc)
```

- Calculate VPN (Virtual Page Number)
- Lookup VPN in TLB
- If lookup was successful
 - Calculate physical address
- If lookup was not successful (i.e., TLB miss)
 - Generate an exception ⇒ *pageFault Exception*

If an exception is generated

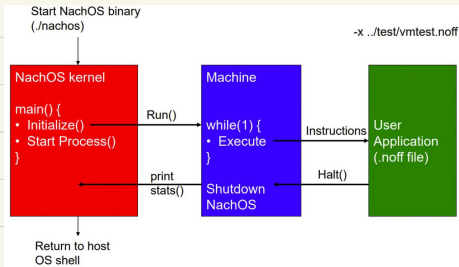
- Call `updateTLB(vpn)`
- Call `Translate(vpn, &phyAddr ...)` again

↳ Instruction is run again to updated T2B

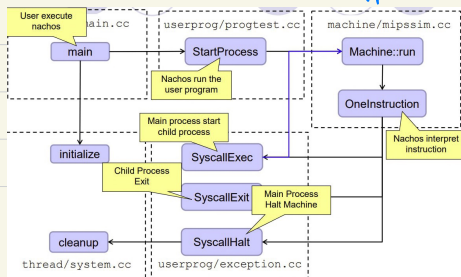
- * Keep track of no. of IBS miss in stats.

Experiment 4:

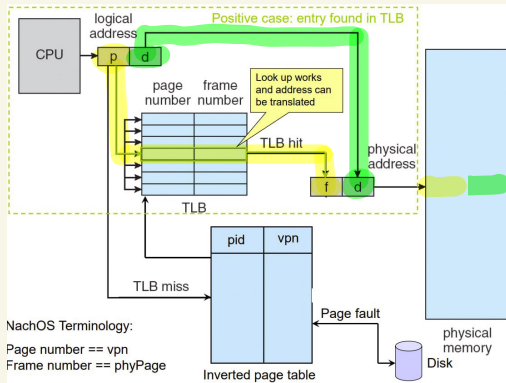
NachWS upon start up:



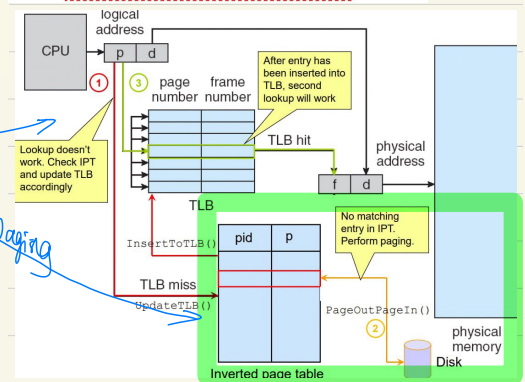
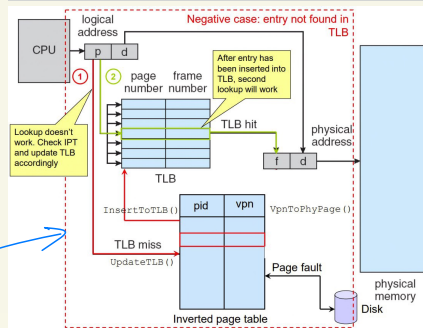
Execution flow of User Application:



TLB hit Case:



machine->tlb - the actual TLB, an array of TranslationEntry object, each notably having a virtualPage and PhysicalPage
 TLBSize - the size of TLB, used mainly when iterating through machine->tlb
 FIFOPointer - the index of TLB which had the earliest entry, ie the entry to be removed
 PageSize - the size of a page and frame
 memoryTable - the IPT, which is stored as an array. The index of the array represents the physical frame number, with each entry having pid, vPage and lastUsed
 NumPhysPages - the size of IPT, and the number of frames in memory. Used mainly when iterating through memoryTable



TLB Miss Case:

Function: UpdateTLB (vpn)
 (defined under vm/tlb.cc)

- Call vpnToPhyPage (vpn)
- If **vpn** can be found in **IPT** (i.e., vpnToPhyPage (vpn) returns a valid phyPage), update TLB
 - Call insertToTLB (vpn, phyPage)
- Otherwise, perform paging and update TLB
 - Call insertToTLB (vpn, PageOutPageIn (vpn))

Function: PageOutPageIn (vpn)

- Determine the victim frame using the **least recently used algorithm**
- Page out victim page
 - Write victim page to swap file
- Page in the new page
 - Load new physical page from swap file
 - Update the IPT table with VPN/PhyPage combination

Task:

Add code to functions in vm/tlb.cc

```
int VpnToPhyPage(int vpn)
void InsertToTLB(int vpn, int phyPage)
int lruAlgorithm(void)
```

Paging:

PageOutPageIn(int vpn)

Defined in vm/tlb.cc. This is called when there is a page fault, meaning that the required memory is not loaded into any physical frames, and has to be loaded in from disk. The frame replaced is determined by the LRU algorithm

- increment stats->NumPageFaults
- phyPage = lruAlgorithm() - this gets the index/frame number for the vpn to replace.
- load the actual memory to the frame
 - call doPageOut(phyPage) - which pages out the frame is memoryTable[phyPage].dirty is TRUE. If we want to see if pages are paged out, we can do so here
 - call doPageIn(vpn, phyPage)
- load respective page into memoryTable[phyPage]. Set memoryTable[phyPage].lastUsed = 0. This will be updated to the correct value in InsertToTLB
- return phyPage, which is then passed into InsertToTLB

OR when there are dirty pages!

Trans-look
imple-
men-
tion

machine->Translate(int virtAddr, int* physAddr, int size, bool writing)

Defined in machine/translate.c. Entry point for memory logic, when the process need to access a certain memory address in its logical space. In exp4, we assume that there is only 2 possible outcomes of this function.

- If virtAddr is in TLB, then the physical address pointer will be assigned
- If virtAddr is not in TLB, then PageFaultException is raised, and the exception handler defined in userprog/exceptions.cc will call UpdateTLB(int vAddr)

The actual implementation:

- split virtAddr into vpn and offset
- iterate through machine->tlb to look for machine->tlb[i].virtualPage == vpn & machine->tlb[i].valid
 - if found, then physAddr will be machine->tlb[i] * PageSize + offset
 - this also means IPT will have this pid, vPage entry, whose lastUsed will be updated. memoryTable[physicalPage].lastUsed = stats->totalTicks
- if not found, return PageFaultException. Somehow, the exception handler will call UpdateTLB(vAddr)

UpdateTLB(int possible_badVAddr)

Defined in vm/tlb.cc. In exp4, we assume that the VAddr is valid, ie within range, and the error is only due to TLB miss, meaning that the VAddr is not found in TLB. The next check is to see if this virtual address is in the IPT.

- split possible_badVAddr into vpn and offset
- phyPage = VpnToPhyPage(vpn) - this function will return the index/frame number for this vpn, or -1 if the vpn is not in the IPT.
 - if found, phyPage >= 0, then call InsertToTLB(vpn, phyPage)
 - if not found, phyPage = -1, then call InsertToTLB(vpn, PageOutPageIn(vpn))
- note that InsertToTLB() is called regardless. Also, stats->NumTlbMisses is not incremented here, but rather in InsertToTLB()

Simple FIFO: if an entry is just inserted, then the entry next to it is the oldest entry

$\text{FIFOPointer} = (i + 1) \% \text{TLBSize}$

1. InsertToTLB(int vpn, int phyPage):

- ↳ Insert VPN & its frame no. into TLB
- ↳ To check all entries in TLB for any invalid entry
- ↳ i points to invalid entry for entry replacement
- ↳ FIFO implementation using FIFO pointer, a static int FIFO pointer
 - ↳ remains in memory forever
 - ↳ No reinitializing

2. VpnToPhyPage():

- Returns physical page for a VPN, if it exists in the ZPT.
- IPT realised as memoryTable data type
 - ↳ Has as many entries as there are physical pages (Constant: NumPhysPages)
- Iterate through memory table to find corresponding physical page entry

Return the index i for the physical page entry for which the following conditions are all true:

- ☐ $\text{memoryTable}[i].\text{valid}$
- ☐ $\text{memoryTable}[i].\text{pid} == \text{currentThread} \rightarrow \text{pid}$
- ☐ $\text{memoryTable}[i].\text{vPage} == \text{vpn}$

Return -1 if no entry can be found that matches the above condition

3. LRU Algorithm:

- Determines which physical page should be paged out
 - ↳ Returns page no. to be paged out
- To find least recently used entry in IPT
 - ↳ Look for physical page stored in $\text{memoryTable}[i].\text{lastUsed}$.

Search for the invalid entry from the beginning of the memoryTable. If there is an invalid entry, return that to be used by the virtual page.

- ☐ $\text{!memoryTable}[i].\text{valid}$

Otherwise, find a victim (entry with smallest lastUsed)

- ☐ Return the page number

```
void InsertToTLB(int vpn, int phyPage)
{
    int i = 0; // entry in the TLB

    //your code to find an empty in TLB or to replace the oldest entry if TLB is full

    print("---- print: InsertToTLB(vpn=xi, phyPage=xi) \n", vpn, phyPage);
    print("v=... i=0 ... \n");
    for (int x = 0; x < TLBSize; x++) {
        print("TLB[i]: vpn=xi, phy=xi, valid=xi \n", x, machine->tlb[x].virtualPage, machine->tlb[x].physicalPage, machine->tlb[x].valid);
    }
    print("---- i=0 ... \n");
    static int FIFOPointer = 0;

    // If TLB has empty entries, that means the TLB is being populated the first time
    for (i = 0; i < TLBSize; i++) {
        if (!machine->tlb[i].valid) {
            break;
        }
    }

    // If no empty page in tlb, then find the oldest one, which is at FIFOPointer
    if (i == TLBSize) {
        i = FIFOPointer;
    }

    FIFOPointer = (i + 1) % TLBSize;
    // copy physical page to memoryTable
    if (machine->tlb[i].valid) {
        memoryTable[machine->tlb[i].physicalPage].dirty = machine->tlb[i].dirty;
        memoryTable[machine->tlb[i].physicalPage].TLBentry = i;
    }

    //update the TLB entry
    machine->tlb[i].virtualPage = vpn;
    machine->tlb[i].physicalPage = phyPage;
    machine->tlb[i].valid = TRUE;
    machine->tlb[i].readOnly = FALSE;
    machine->tlb[i].use = FALSE;
    machine->tlb[i].dirty = memoryTable[phyPage].dirty;

    //update the corresponding memoryTable
    memoryTable[phyPage].TLBentry = i;
    DEBUG("p", "The corresponding TLBentry for Page xi in TLB is xi \n", vpn, i);
    //reset lastUsed to current ticks since it is being used at this moment.
    //for the implementation of LRU algorithm.
    memoryTable[phyPage].lastUsed = stats->totalTicks;

    //increase the number of tlb misses
    stats->numTlbMisses++;
}

//updateZPT(int vpn)
{
    //your code here to get a physical frame for the page vpn
    //you can refer to PopulatePage(int vpn) to see how an entry was created in ipt
    for (int i = 0; i < NumPhysPages; i++) {
        DEBUG("p", "...IPT xi, pid, vpn, last used and valid is xi xi xi xi \n", i, memoryTable[i].pid,
            memoryTable[i].vPage, memoryTable[i].lastUsed, memoryTable[i].valid);
    }
    //if not in TLB, then check page table
    for (int i = 0; i < NumPhysPages; i++) {
        if (memoryTable[i].valid && memoryTable[i].pid == currentThread->pid && memoryTable[i].vPage == vpn) {
            DEBUG("p", "...found PPT xi, pid, vpn, last used and valid is xi xi xi xi \n", i, memoryTable[i].pid, memoryTable[i].vPage, memoryTable[i].lastUsed, memoryTable[i].valid);
            return i;
        }
    }

    // else return -1
    return -1;
}
```

```
int lruAlgorithm(void)
{
    //your code here to find the physical frame that should be freed
    //according to the LRU algorithm.
    int phyPage = 0;

    int leastRecentlyUsedTick = 2147483647;

    // If empty page, then return that
    // Else find time with the smallest tick (referenced long time ago)
    for (int i = 0; i < NumPhysPages; i++) {
        if (!memoryTable[i].valid) {
            phyPage = i;
            break;
        }
        if (memoryTable[i].lastUsed < leastRecentlyUsedTick) {
            leastRecentlyUsedTick = memoryTable[i].lastUsed;
            phyPage = i;
        }
    }

    return phyPage;
}
```

To get CSV answers [🔗](#)

For every TLB miss, we want to know what the IPT and TLB entry is being updated. The best place to do so is in `InsertInTLB(vpn, phyPage)`, since it will be called everytime a TLB miss happens. We can print the state of `machine->tlb, memoryTable` to get the values of the tables, and `phyPage` to get the updated IPT entry, and `FIFOPointer` to get the updated TLB entry.

If we want to get the data from only Page faults, we can do so in `PageOutPageIn()`.

To get the entries where pages are paged out, we can print `phyPage` in `DoPageOut(phyPage)` under the condition that `memoryTable[phyPage].dirty` is `TRUE`